Táblák particionálása

A particionálás célja: oszd meg és uralkodj! Ha a táblát egy mező mentén particionáljuk:

- Gyorsulnak a lekérdezések, ha az egyes partíciók más-más fizikai adattárolón vannak, mert párhuzamosíthatók.
- Egyes lekérdezésekhez elég lehet néhány partíció, nem kell az összes.
- A nagy táblák feltöltési ideje gyakran korlátozott, például banki rendszerben éjszakánként stb., olyan időre mikor az adatok nem változnak. Az INSERT és DELETE logolt utasítások (implicit, logolt tranzakciók), ezért egyszerre túl nagy mennyiség feltöltése vagy törlése lassú (pl. az indexeket újra kell építeni utána). Particionálás esetén gyorsabbá válik a feltöltés, mert a nagy tábla insert helyett egy üres tábla-partícióba írunk, ami egy gyors 'minimally logged' művelet (simple recovery módban).

Tipikus particionálási kulcs a dátum, pl. havonként vagy évenként. A tábla oszloptár-indexét, ha van neki, szintén particionálni kell (illesztett index, aligned index). Ekkor a partíció-váltás (partition switching) nem igényli az index újraépítését, csak meta-adat változik. Az SQL Server lényeges particionálási fogalmai:

- **Particionálási séma**: a partíciók fájlcsoportokhoz rendelése
- **Particionálási függvény**: az egyes rekordokat a particionálás alapjául szolgáló mező(k), például a dátum alapján valamelyik partícióhoz rendeli hozzá

A partíciók mérete (rekordszáma) különböző lehet!

Az oszloptár indexről

Nagy méretű, sok mezős, ritkán változó adattárház-táblák számára ajánlott. A tábla blokkjaiban az egy rekordhoz tartozó mezők nem együtt vannak tárolva, hanem az **egyes mezők külön-külön**, és az egy rekordhoz tartozó értékek össze vannak linkelve. Gyorsaságát a kevés I/O műveletnek (blokkolvasásnak) köszönheti, mely a lapok tömörítésének (csökken a méret -> kevesebb blokk) és annak köszönhető, hogy a lekérdezésekhez általában nem kell az egész rekordot beolvasni, csak a szükséges mezőket. Ez különösen akkor szembetűnő, ha a táblának **nagyon sok mezője van**, ami tipikus az adattárházak esetén. A tömörítés különösen akkor hatékony, ha egy-egy mező értékei eléggé hasonlóak. Az ingyenesen elérhető Postgres jelenleg még nem támogatja[1], csak az SQL Server.

Tervezési minta particionálásra és nagy tábla importálására minimális logolással

Hozzunk létre egy particionált táblát, illesztett oszloptár-indexszel, és töltsünk be egy új partíciót (például egy új évnyi adatot). A lépések:

1. A particionálási séma, függvény létrehozása
2. Meglévő adatok betöltése
3. Az új év betöltéséhez a táblával azonos sémájú (és tömörítésű) üres segédtáblát készítünk
4. Ebben check constraint védi a particionálási kulcsot, hogy ne kerülhessen bele téves adat
5. Betöltjük az új adatokat a segédtáblába és oszloptár-indexet készítünk hozzá
6. Partíció-cserével betöltjük a segédtáblát a ténytábla következő üres partíciójába

[1] https://swarm64.com/post/postgresql-columnstore-index-intro/

7. A következő partíció betöltése előtt töröljük az oszloptár-indexet és értelemszerűen átírjuk a check constraint-et

DEMO: Particionálás és partíciók betöltése az AdventureworksDW2016CTP3adatbázisban

```
----először tervezünk: év szerint akarjuk particionálni a ténytáblát:
select min(OrderDate), max(OrderDate) FROM AdventureworksDW2016CTP3.dbo.FactInternetSales --2010..2014
--tehát 5 partíciót tervezünk az 5 évnek
--a korábbi táblát (ha volt) töröljük
go
drop table InternetSales
go
--part. függvény 5 partícióra:
CREATE PARTITION FUNCTION PfInternetSalesYear (TINYINT) AS RANGE LEFT FOR VALUES (10, 11, 12, 13)
go
-- a 4 határértékből 5 partíció lesz
--pl. '10': az év száma <= 2010 a LEFT miatt
--a TINYINT csak 1 bájtos, 0..255
--part. séma: minden partíció ugyanoda megy:
CREATE PARTITION SCHEME PsInternetSalesYear AS PARTITION PfInternetSalesYear ALL TO ([PRIMARY])
go
--a táblába betesszük a part. mezőt (tinyint), egyúttal a kulcsát is módosítjuk, mert az identity
--már csak partíción belül lesz egyedi a partíció-cserék miatt
drop table InternetSales
go
CREATE TABLE dbo.InternetSales(
        InternetSalesKey INT NOT NULL IDENTITY(1,1),
        PcInternetSalesYear TINYINT NOT NULL,  --itt az új mező
        CustomerDwKey INT,
        ProductKey INT NOT NULL,
        DateKey INT NOT NULL,
        OrderQuantity SMALLINT NOT NULL DEFAULT 0,
        SalesAmount MONEY NOT NULL DEFAULT 0,
        UnitPrice MONEY NOT NULL DEFAULT 0,
        DiscountAmount FLOAT NOT NULL DEFAULT 0,
        CONSTRAINT PK_InternetSales PRIMARY KEY (InternetSalesKey, PcInternetSalesYear)
)
ON PsInternetSalesYear(PcInternetSalesYear) --ezzel particionáljuk
GO
--lap-tömörítés
ALTER TABLE dbo.InternetSales REBUILD WITH (DATA_COMPRESSION = PAGE);
GO
--feltöltjük az adatokat 2013-ig
INSERT INTO dbo.InternetSales (PcInternetSalesYear, CustomerDwKey, ProductKey, DateKey,
OrderQuantity, SalesAmount, UnitPrice, DiscountAmount)
SELECT CAST(SUBSTRING(CAST(FIS.OrderDateKey AS CHAR(8)), 3, 2) AS TINYINT) AS PcInternetSalesYear,
--itt van egy trükk: a mező int típusú, hogy kicsi legyen, azonban dátumnak olvasható int-ek vannak benne,
pl. 20110223
        null,
        FIS.ProductKey, FIS.OrderDateKey, FIS.OrderQuantity, FIS.SalesAmount,
        FIS.UnitPrice, FIS.DiscountAmount
FROM AdventureworksDW2016CTP3.dbo.FactInternetSales AS FIS
WHERE CAST(SUBSTRING(CAST(FIS.OrderDateKey AS CHAR(8)), 3, 2) AS TINYINT) < 13
GO
--melyik partícióba hány rekord került?
SELECT
    partition_number,
    row_count
FROM sys.dm_db_partition_stats
WHERE object_id = OBJECT_ID('InternetSales');
--vagy:
SELECT $PARTITION.PfInternetSalesYear(PcInternetSalesYear), COUNT(*) AS NumberOfRows
FROM dbo.InternetSales GROUP BY $PARTITION.PfInternetSalesYear(PcInternetSalesYear)
--eredmény:
PartitionNumber  NumberOfRows
1                               14              --2010
2                               2216    --2011
3                               3397    --2012
```

```
--a negyedik még üres a szűrés miatt

--oszloptár-index az öszes mezőre:
CREATE COLUMNSTORE INDEX CSI_InternetSales ON dbo.InternetSales
        (InternetSalesKey, PcInternetSalesYear, CustomerDwKey, ProductKey, DateKey,
        OrderQuantity, SalesAmount, UnitPrice, DiscountAmount)
ON PsInternetSalesYear(PcInternetSalesYear) --figyelem: itt illesztjük az indexet a partíciókhoz
GO

--A 2013-as évből az új partíció adatait egy azonos szerkezetű táblában fogadjuk majd.
--Hogy semmiképp se kerülhessen bele más évből származó adat, egy check constraint-tal védjük:
create TABLE dbo.InternetSalesNew (
        InternetSalesKey INT NOT NULL IDENTITY(1,1),
        PcInternetSalesYear TINYINT NOT NULL CHECK (PcInternetSalesYear = 13), --ITT A VÉDELEM
        CustomerDwKey INT,
        ProductKey INT NOT NULL,
        DateKey INT NOT NULL,
        OrderQuantity SMALLINT NOT NULL DEFAULT 0,
        SalesAmount MONEY NOT NULL DEFAULT 0,
        UnitPrice MONEY NOT NULL DEFAULT 0,
        DiscountAmount FLOAT NOT NULL DEFAULT 0,
        CONSTRAINT PK_InternetSalesNew PRIMARY KEY (InternetSalesKey, PcInternetSalesYear)
)
GO
--minden beállítása ugyanaz:
--go
ALTER TABLE dbo.InternetSalesNew REBUILD WITH (DATA_COMPRESSION = PAGE)
GO
--most betöltjük a 2013-as évet
--a trükk az, hogy mivel a tábla üres volt, az insert nem lesz logolva, tehát gyorsabb lesz
INSERT INTO dbo.InternetSalesNew (PcInternetSalesYear, CustomerDwKey,ProductKey, DateKey,
        OrderQuantity, SalesAmount,UnitPrice, DiscountAmount)
SELECT CAST(SUBSTRING(CAST(FIS.OrderDateKey AS CHAR(8)), 3, 2) AS TINYINT) AS PcInternetSalesYear,
        null, FIS.ProductKey, FIS.OrderDateKey,FIS.OrderQuantity, FIS.SalesAmount,
        FIS.UnitPrice, FIS.DiscountAmount
FROM AdventureworksDW2016CTP3.dbo.FactInternetSales AS FIS
WHERE CAST(SUBSTRING(CAST(FIS.OrderDateKey AS CHAR(8)), 3, 2) AS TINYINT) = 13
GO
--a betöltő-táblára is elkészítjük az oszloptár-indexet
CREATE COLUMNSTORE INDEX CSI_InternetSalesNew
ON dbo.InternetSalesNew (InternetSalesKey, PcInternetSalesYear, CustomerDwKey, ProductKey, DateKey,
OrderQuantity, SalesAmount, UnitPrice, DiscountAmount);
GO
--most betesszük a betöltő táblát a ténytábla 4. partíciójába:
ALTER TABLE dbo.InternetSalesNew SWITCH TO dbo.InternetSales PARTITION 4
--ez csak metaadat-átírást jelentett!
--lássuk a partíciókat:
SELECT $PARTITION.PfInternetSalesYear(PcInternetSalesYear) AS PartitionNumber, COUNT(*) AS NumberOfRows
FROM dbo.InternetSales GROUP BY $PARTITION.PfInternetSalesYear(PcInternetSalesYear)
order by PartitionNumber

PartitionNumber  NumberOfRows
1                               14
2                               2216
3                               3397
4                               52801

select top 100 * from InternetSalesNew --üres!
--a következő partíció betöltése előtt: törölni az oszloptár-indexet és átírni a check constraint-et!
```

Hasonlóképpen valósítható meg egy segédtáblán végzett minimálisan logolt TRUNCATE TABLE művelettel egy particionált tábla már korábban (esetleg hibásan) betöltött partíciójának a törlése is.

Megjegyzés: a particionálás és a tömörítés elvégezhető az SSMS GUI-n is a tábla Storage menüjében. Ha olyan scriptet akarunk, amely a tábla particionálását is tartalmazza, akkor kapcsoljuk be a Dependent Objects opciót az adatbázis scriptgenerátorában!

ÖNÁLLÓ FELADAT:

Másoljuk át az AdventureworksDW2016CTP3.FactResellerSales táblát a saját adatbázisunkba és particionáljuk a DueDateKey szerint! Készítsünk egy scriptet, amely elvégzi egy partíció betöltését minimálisan logolt INSERT utasításokkal!